

THREAT AND RISK ANALYSIS

Bank of Anthos

Anthos Bank Investment Ltd.

Version	1.1
Date	2020-07-02
Authors	Ferenc Bator

Scope and Assumptions

System Description

The Data Flow Diagram below provides an overview of the analyzed architecture.

```

flowchart TD
    unauthorized-external-agent["TA1"]
    authorized-employee["TA2"]
    unauthorized-insider["TA3"]
    browser["User"]
    developer_client["DevOps"]
    waf["WAF"]
    ingress["istio-ingress-gateway"]
    frontend["frontend"]
    user_service["User Service"]
    contacts["Contacts"]
    user_service["User Service"]
    accounts_db["accounts-db"]
    ledger_writer["ledger writer"]
    balance_reader["balance reader"]
    trans_history["trans history"]
    ledger_db["ledger-db"]
    kms["KMS"]
    hsm["HSM"]
    logging["Cloud Logging"]
    monitoring["Cloud Monitoring"]
    subgraph Internet ["Internet"]
        unauthorized-external-agent
        subgraph Google_Cloud_Platform ["Google Cloud Platform"]
            unauthorized-external-agent
            subgraph project_boa_gke ["project boa-gke"]
                subgraph VPC ["VPC"]
                    subgraph public_subnet ["public subnet"]
                        waf
                    end
                end
                subgraph privat_subnet ["privat subnet"]
                    subgraph GKE ["GKE"]
                        subgraph istion_system ["istion-system"]
                            ingress
                        end
                    end
                    subgraph frontend ["frontend"]
                    end
                    subgraph accounts ["accounts"]
                        user_service
                        contacts
                    end
                    subgraph transactions ["transactions"]
                        ledger_writer
                        balance_reader
                        trans_history
                    end
                end
            end
        end
    end
    subgraph project_boa_sql ["project boa-sql"]
        accounts_db
        ledger_db
    end
    subgraph project_boa_sec ["project boa-sec"]
        kms
    end
    subgraph project_boa_ops ["project boa-ops"]
        logging
        monitoring
    end
end
browser
end
subgraph Company_Network ["Company Network"]
    authorized-employee
    unauthorized-insider
    developer_client
    hsm
end

```

```

browser -->|https / Link to waf| waf
developer_client -->|https / Link to waf| waf
waf -->|https / Link to app| ingress
ingress -->|https| frontend
frontend -->|https| user_service
frontend -->|https| contacts
frontend -->|https| trans_history
frontend -->|https| balance_reader
frontend -->|https| ledger_writer
user_service -->|sql_encrypted / Link to database| accounts_db
contacts -->|sql_encrypted / Link to database| accounts_db
user_service -->|sql_encrypted / Link to database| accounts_db
ledger_writer -->|sql_encrypted / Link to database| ledger_db
balance_reader -->|sql_encrypted / Link to database| ledger_db
trans_history -->|sql_encrypted / Link to database| ledger_db
hsm -->|kek / key transfer| kms

```

Trust Boundaries

Name	Technology	Description
Internet	internet	Internet
Google Cloud Platform	internet	Google Cloud
project boa-gke	project	Google Project
VPC	vpc	Virtual Private Cloud
public subnet	subnet	internet facing zone
privat subnet	subnet	Application Network
GKE	kubernetes-cluster	kubernetes cluster
frontend	kubernetes-network-policies	kubernetes frontend namespace
accounts	kubernetes-network-policies	kubernetes default namespace
transactions	kubernetes-network-policies	kubernetes default namespace
istion-system	kubernetes-network-policies	kubernetes istio-system
project boa-sql	cloud-services	Azure Shared Services
project boa-sec	project	Contains Secret Manager and KMS instances for secrets that are specific to the Bank of Anthos application.
project boa-ops	project	Used for storing environment logs as well as monitoring the environment instance of the Bank of Anthos application.
Company Network	on-premise	trusted on-premise company network

Technical Assets

Name	Technology	Description
User	browser	The browser used by the end customer
DevOps	devops-client	laptop used by developers and operators to manage the system
WAF	waf	Google Cloud Armor Web Application Firewall
istio-ingress-gateway	kubernetes-ingress	ISTIO ingress gateway
frontend	kubernetes-pod	Exposes an HTTP server to serve the website. Contains a login page, a signup page, and a home page.
User Service	kubernetes-pod	
Contacts	kubernetes-pod	Stores a list of additional accounts that are associated with a user. These accounts are

		listed in the application's Send Payment and Deposit forms.
User Service	kubernetes-pod	Manages user accounts and authentication. The service signs JWTs that are used for authentication by other services.
accounts-db	database-sql	SQL database
ledger writer	kubernetes-pod	Accepts and validates incoming transactions before writing them to the ledger.
balance reader	kubernetes-pod	Provides an efficient readable cache of user balances, as read from ledger-db.
trans history	kubernetes-pod	Provides an efficient readable cache of past transactions, as read from ledger-db.
ledger-db	database-nosql	CloudSQL for hyperledger
KMS	hsm	Google Key Management Service
HSM	hsm	on-premise Hardware Security Module (HSM)
Cloud Logging	logging	Cloud Monitoring
Cloud Monitoring	monitoring	KMS

Problem Description

Data Assets

Name	Description	C	I	A
A1	Angular and other client-side code delivered by the application.	Public	Public	Public
A2	OIDC identity token	Internal	Public	Public
A3	OAuth2 access token	Internal	Public	Public
A4	OAuth2 refresh token	Internal	Public	Public
A5	active/ongoing bank transactions	Restricted	Internal	Internal
A6	bank transaction history	Restricted	Internal	Internal
A7	account balance of client	Restricted	Internal	Internal
A8	root certificates	Restricted	Internal	Internal
A9	Session ID	Internal	Internal	Internal
A10	personal-related end user information like e-mail to identify user	Unknown	Unknown	Unknown
A11	password of a user	Restricted	Internal	Internal
A12	contacts of the customer from past transactions	Internal	Internal	Internal
A13	credentials to access ledger db	Restricted	Internal	Internal
A14	credentials to access ledger db	Restricted	Internal	Internal

Threat Agents

Name	Description
TA1	external threat agent

TA2	authorized employee
TA3	Employee/Contractor of the company that is not authorized to access the system

Risk Assessment

Risk Matrix

We follow the [OWASP Risk Rating Methodology](#).

Overall Risk Severity = Impact x Likelihood			
Impact ↓ Likelihood →	Low	Medium	High
	Low	Medium	High
	Low	Medium	Critical
Likelihood (columns)	LOW	MEDIUM	HIGH

Identified Risks

ID	Likelihood	Impact	Severity	Risk	Action	Mitigation	Res. Impact	Res. Likelihood
container-baseimage-backdooring@balance_reader	LOW(1)	MEDIUM(2)	LOW(2)	CWE-0 Container Base Image Backdooring: asset 'balance_reader' has risk of container image backdooring through base images When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors	MITIGATE	Google distroless image is used to mitigate risks. The product owner accepted the residual risks.	MEDIUM(2)	LOW(1)
container-baseimage-backdooring@contacts	LOW(1)	MEDIUM(2)	LOW(2)	CWE-0 Container Base Image Backdooring: asset 'contacts' has risk of container image backdooring through base images When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable	MITIGATE	Google distroless image is used to mitigate risks. The product owner accepted the residual risks.	MEDIUM(2)	LOW(1)

				components or backdoors				
container-baseimage-backdooring@frontend	LOW(1)	MEDIUM(2)	LOW(2)	CWE-0 Container Base Image Backdooring: asset 'frontend' has risk of container image backdooring through base images When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors	MITIGATE	Google distroless image is used to mitigate risks. The product owner accepted the residual risks.	MEDIUM(2)	LOW(1)
container-baseimage-backdooring@ledger_writer	LOW(1)	MEDIUM(2)	LOW(2)	CWE-0 Container Base Image Backdooring: asset 'ledger_writer' has risk of container image backdooring through base images When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors	MITIGATE	Google distroless image is used to mitigate risks. The product owner accepted the residual risks.	MEDIUM(2)	LOW(1)
container-baseimage-backdooring@trans_history	LOW(1)	MEDIUM(2)	LOW(2)	CWE-0 Container Base Image Backdooring: asset 'trans_history' has risk of container image backdooring through base images When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors	MITIGATE	Google distroless image is used to mitigate risks. The product owner accepted the residual risks.	MEDIUM(2)	LOW(1)

container-baseimage-backdooring@user_service	LOW(1)	MEDIUM(2)	LOW(2)	CWE-0 Container Base Image Backdooring: asset 'user_service' has risk of container image backdooring through base images When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors	MITIGATE	Google distroless image is used to mitigate risks. The product owner accepted the residual risks.	MEDIUM(2)	LOW(1)
cross-site-request-forgery@balance_reader	HIGH(3)	MEDIUM(2)	HIGH(6)	CWE-0 Cross-Site Request Forgery (CSRF): asset 'balance_reader' has risk of Cross Site Request Forgery(CSRF) When a web application is accessed via web protocols Cross-Site Request Forgery (CSRF) risks might arise.	TBD	Try to use anti-CSRF tokens of the double-submit patterns (at least for logged-in requests). When your authentication scheme depends on cookies (like session or token cookies), consider marking them with the same-site flag. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level	TBD(-1)	TBD(-1)
cross-site-request-forgery@contacts	HIGH(3)	MEDIUM(2)	HIGH(6)	CWE-0 Cross-Site Request Forgery (CSRF): asset 'contacts' has risk of Cross Site Request Forgery(CSRF) When a web application is accessed via web protocols Cross-Site Request	TBD	Try to use anti-CSRF tokens of the double-submit patterns (at least for logged-in requests). When your authentication scheme depends on cookies (like	TBD(-1)	TBD(-1)

				Forgery (CSRF) risks might arise.		session or token cookies), consider marking them with the same-site flag. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level		
cross-site-request-forgery@frontend	HIGH(3)	MEDIUM(2)	HIGH(6)	CWE-0 Cross-Site Request Forgery (CSRF): asset 'frontend' has risk of Cross Site Request Forgery(CSRF) When a web application is accessed via web protocols Cross-Site Request Forgery (CSRF) risks might arise.	TBD	Try to use anti-CSRF tokens of the double-submit patterns (at least for logged-in requests). When your authentication scheme depends on cookies (like session or token cookies), consider marking them with the same-site flag. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level	TBD(-1)	TBD(-1)
cross-site-request-forgery@ledger_writer	HIGH(3)	MEDIUM(2)	HIGH(6)	CWE-0 Cross-Site Request Forgery (CSRF): asset 'ledger_writer' has risk of Cross Site Request Forgery(CSRF) When a web application is accessed via web protocols Cross-Site Request Forgery (CSRF) risks might arise.	TBD	Try to use anti-CSRF tokens of the double-submit patterns (at least for logged-in requests). When your authentication scheme depends on cookies (like session or token	TBD(-1)	TBD(-1)

						cookies), consider marking them with the same-site flag. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level		
cross-site-request-forgery@trans_history	HIGH(3)	MEDIUM(2)	HIGH(6)	CWE-0 Cross-Site Request Forgery (CSRF): asset 'trans_history' has risk of Cross Site Request Forgery(CSRF) When a web application is accessed via web protocols Cross-Site Request Forgery (CSRF) risks might arise.	TBD	Try to use anti-CSRF tokens of the double-submit patterns (at least for logged-in requests). When your authentication scheme depends on cookies (like session or token cookies), consider marking them with the same-site flag. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level	TBD(-1)	TBD(-1)
cross-site-request-forgery@user_service	HIGH(3)	MEDIUM(2)	HIGH(6)	CWE-0 Cross-Site Request Forgery (CSRF): asset 'user_service' has risk of Cross Site Request Forgery(CSRF) When a web application is accessed via web protocols Cross-Site Request Forgery (CSRF) risks might arise.	TBD	Try to use anti-CSRF tokens of the double-submit patterns (at least for logged-in requests). When your authentication scheme depends on cookies (like session or token cookies), consider	TBD(-1)	TBD(-1)

						marking them with the same-site flag. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level		
software-integrity@balance_reader	MEDIUM(2)	VERY HIGH(4)	HIGH(8)	CWE-0 Software Integrity: asset 'balance_reader' lacks software integrity verification (signing, SBOM, etc.) Technical assets should verify software integrity (signing, SBOM, etc.).	TBD	Verify software integrity through signing and SBOM.	TBD(-1)	TBD(-1)
software-integrity@contacts	MEDIUM(2)	VERY HIGH(4)	HIGH(8)	CWE-0 Software Integrity: asset 'contacts' lacks software integrity verification (signing, SBOM, etc.) Technical assets should verify software integrity (signing, SBOM, etc.).	TBD	Verify software integrity through signing and SBOM.	TBD(-1)	TBD(-1)
software-integrity@frontend	MEDIUM(2)	VERY HIGH(4)	HIGH(8)	CWE-0 Software Integrity: asset 'frontend' lacks software integrity verification (signing, SBOM, etc.) Technical assets should verify software integrity (signing, SBOM, etc.).	TBD	Verify software integrity through signing and SBOM.	TBD(-1)	TBD(-1)
software-integrity@ledger_writer	MEDIUM(2)	VERY HIGH(4)	HIGH(8)	CWE-0 Software Integrity: asset 'ledger_writer' lacks software integrity verification (signing, SBOM, etc.) Technical assets should verify software integrity	TBD	Verify software integrity through signing and SBOM.	TBD(-1)	TBD(-1)

				(signing, SBOM, etc.).				
software-integrity@trans_history	MEDIUM(2)	VERY HIGH(4)	HIGH(8)	CWE-0 Software Integrity: asset 'trans_history' lacks software integrity verification (signing, SBOM, etc.) Technical assets should verify software integrity (signing, SBOM, etc.).	TBD	Verify software integrity through signing and SBOM.	TBD(-1)	TBD(-1)
software-integrity@user_service	MEDIUM(2)	VERY HIGH(4)	HIGH(8)	CWE-0 Software Integrity: asset 'user_service' lacks software integrity verification (signing, SBOM, etc.) Technical assets should verify software integrity (signing, SBOM, etc.).	TBD	Verify software integrity through signing and SBOM.	TBD(-1)	TBD(-1)
vulnerable-dependencies@balance_reader	MEDIUM(2)	HIGH(3)	HIGH(6)	CWE-0 Vulnerable Dependencies: asset 'balance_reader' uses vulnerable dependencies without scanning Technical assets should scan dependencies for known vulnerabilities.	TBD	Regularly scan dependencies for known vulnerabilities.	TBD(-1)	TBD(-1)
vulnerable-dependencies@contacts	MEDIUM(2)	HIGH(3)	HIGH(6)	CWE-0 Vulnerable Dependencies: asset 'contacts' uses vulnerable dependencies without scanning Technical assets should scan dependencies for known vulnerabilities.	TBD	Regularly scan dependencies for known vulnerabilities.	TBD(-1)	TBD(-1)

vulnerable-dependencies@frontend	MEDIUM(2)	HIGH(3)	HIGH(6)	CWE-0 Vulnerable Dependencies: asset 'frontend' uses vulnerable dependencies without scanning Technical assets should scan dependencies for known vulnerabilities.	TBD	Regularly scan dependencies for known vulnerabilities.	TBD(-1)	TBD(-1)
vulnerable-dependencies@ledger_writer	MEDIUM(2)	HIGH(3)	HIGH(6)	CWE-0 Vulnerable Dependencies: asset 'ledger_writer' uses vulnerable dependencies without scanning Technical assets should scan dependencies for known vulnerabilities.	TBD	Regularly scan dependencies for known vulnerabilities.	TBD(-1)	TBD(-1)
vulnerable-dependencies@trans_history	MEDIUM(2)	HIGH(3)	HIGH(6)	CWE-0 Vulnerable Dependencies: asset 'trans_history' uses vulnerable dependencies without scanning Technical assets should scan dependencies for known vulnerabilities.	TBD	Regularly scan dependencies for known vulnerabilities.	TBD(-1)	TBD(-1)
vulnerable-dependencies@user_service	MEDIUM(2)	HIGH(3)	HIGH(6)	CWE-0 Vulnerable Dependencies: asset 'user_service' uses vulnerable dependencies without scanning Technical assets should scan dependencies for known vulnerabilities.	TBD	Regularly scan dependencies for known vulnerabilities.	TBD(-1)	TBD(-1)

Methodology

STRIDE

Likelihood Scale

Impact Scale

About Taralizer

Risk rules checked by Taralizer

OWASP Application Security Verification Standard — 4.0.2

The OWASP Application Security Verification Standard is specified [HERE](#).

The following list provides supported rules:

Rule missing-authentication

Title	Missing Authentication
Description	Technical assets should authenticate incoming requests.
CWE	0
Mitigation	apply an authentication method to the technical asset.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule missing-authorization

Title	Missing Authorization
Description	Technical assets should perform authorization checks on incoming requests.
CWE	0
Mitigation	apply an authorization check to the technical asset.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule insecure-proto

Title	Unencrypted Communication
Description	Data at transition should be encrypted.
CWE	0
Mitigation	apply an authentication method to the technical asset.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule missing-vault

Title	Missing Vault (Secret Storage)
Description	In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.
CWE	0
Mitigation	Consider using a Vault (Secret Storage) to securely store and access config secrets (like credentials, private keys, client certificates, etc.)
URL	https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule missing-waf

Title	Missing Web Application Firewall (WAF)
Description	To have a first line of filtering defense, security architectures with web-services or web-applications should include a WAF in front of them. Even though a WAF is not a replacement for security (all components must be secure even without a WAF) it adds another layer of defense to the overall system by delaying some attacks and having easier attack alerting through it
CWE	0
Mitigation	Consider placing a fully-managed Web Application Firewall (WAF) in front of the web-services and/or web-applications
URL	https://cheatsheetseries.owasp.org/cheatsheets/Virtual_Patching_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule cross-site-scripting

Title	Cross-Site Scripting (XSS)
Description	For each web application Cross-Site Scripting (XSS) risks might arise. In terms of the overall risk level take other applications running on the same domain into account as well.
CWE	0
Mitigation	Try to encode all values sent back to the browser and also handle DOM-manipulations in a safe way to avoid DOM-based XSS. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_scripting_Prevention_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule container-baseimage-backdooring

Title	Container Base Image Backdooring
Description	When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors
CWE	0
Mitigation	Apply hardening of all container infrastructures (see for example the CIS-Benchmarks for Docker and Kubernetes and the Docker Bench for Security Use only trusted base images of the original vendors, verify digital signatures and apply image creation best practices. Also consider using Google's Distrosless base images or otherwise very small base images. Regularly execute container image scans with tools checking the layers for vulnerable components.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule cross-site-request-forgery

Title	Cross-Site Request Forgery (CSRF)
Description	When a web application is accessed via web protocols Cross-Site Request Forgery (CSRF) risks might arise.
CWE	0
Mitigation	Try to use anti-CSRF tokens of the double-submit patterns (at least for logged-in requests). When your authentication scheme depends on cookies (like session or token cookies), consider marking them with the same-site flag. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level

URL	https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule missing-audit-logging

Title	Missing Audit Logging
Description	Technical assets should log authentication and authorization events for security monitoring.
CWE	0
Mitigation	Implement audit logging for authentication and authorization events.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Audit_Logging_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule missing-input-validation

Title	Missing Input Validation
Description	Web applications should validate all user-supplied input to prevent injection attacks.
CWE	0
Mitigation	Apply input validation to all user-supplied data.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule missing-access-control

Title	Missing Access Control
Description	Web applications should enforce proper access control checks on all resources.
CWE	0
Mitigation	Implement proper access control checks on all resources.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule missing-cryptographic-controls

Title	Missing Cryptographic Controls
Description	Technical assets should use cryptographic controls for sensitive data.
CWE	0
Mitigation	Apply cryptographic controls for sensitive data.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Encryption_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule missing-injection-protection

Title	Missing Injection Protection
--------------	------------------------------

Description	Web applications should protect against injection attacks (SQL, command, NoSQL, LDAP).
CWE	0
Mitigation	Apply input validation and parameterized queries.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule security-misconfiguration

Title	Security Misconfiguration
Description	Technical assets should have proper security configuration (security headers, weak defaults).
CWE	0
Mitigation	Apply proper security configuration.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Server_Security_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule vulnerable-dependencies

Title	Vulnerable Dependencies
Description	Technical assets should scan dependencies for known vulnerabilities.
CWE	0
Mitigation	Regularly scan dependencies for known vulnerabilities.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Dependency_Usage_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule authentication-failure

Title	Authentication Failure
Description	Technical assets should implement proper authentication for all endpoints.
CWE	0
Mitigation	Implement proper authentication for all endpoints.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule software-integrity

Title	Software Integrity
Description	Technical assets should verify software integrity (signing, SBOM, etc.).
CWE	0
Mitigation	Verify software integrity through signing and SBOM.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Software_Integrity_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule insufficient-logging

Title	Insufficient Logging and Monitoring
Description	Technical assets should monitor and alert for security events.
CWE	0
Mitigation	Implement monitoring and alerting for security events.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule ssrf-vulnerability

Title	SSRF Vulnerability
Description	Technical assets should protect against Server-Side Request Forgery attacks.
CWE	0
Mitigation	Protect against SSRF for URL fetching.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule http-security-headers

Title	HTTP Security Headers
Description	Web applications should enforce security headers (HSTS, CSP, X-Frame-Options, etc.).
CWE	0
Mitigation	Enforce security headers.
URL	https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule missing-external-system-review

Title	Missing External System Review
Description	Technical assets should review security of external systems they communicate with.
CWE	0
Mitigation	Review security of external systems.
URL	https://cheatsheetseries.owasp.org/cheatsheets/Third_Party_Component_Security_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Rule session-management

Title	Session Management
Description	Technical assets should implement secure session management (session fixation prevention, secure cookie flags, etc.).
CWE	0
Mitigation	Implement secure session management.

URL	https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
Base Likelihood	NONE(0)
Base Impact	NONE(0)

Disclaimer

Ferenc Bator conducted this threat analysis using the open-source TARALIZER toolkit on the applications and systems that were modeled as of this report's date. Information security threats are continually changing, with new vulnerabilities discovered on a daily basis, and no application can ever be 100% secure no matter how much threat modeling is conducted. It is recommended to execute threat modeling and also penetration testing on a regular basis (for example yearly) to ensure a high ongoing level of security and constantly check for new attack vectors. This report cannot and does not protect against personal or business loss as the result of use of the applications or systems described. Ferenc Bator and the TARALIZER toolkit offers no warranties, representations or legal certifications concerning the applications or systems it tests. All software includes defects: nothing in this document is intended to represent or warrant that threat modeling was complete and without error, nor does this document represent or warrant that the architecture analyzed is suitable to task, free of other defects than reported, fully compliant with any industry standards, or fully compatible with any operating system, hardware, or other application. Threat modeling tries to analyze the modeled architecture without having access to a real working system and thus cannot and does not test the implementation for defects and vulnerabilities. These kinds of checks would only be possible with a separate code review and penetration test against a working system and not via a threat model. By using the resulting information you agree that John Doe and the Threagile toolkit shall be held harmless in any event. This report is confidential and intended for internal, confidential use by the client. The recipient is obligated to ensure the highly confidential contents are kept secret. The recipient assumes responsibility for further distribution of this document. In this particular project, a timebox approach was used to define the analysis effort. This means that the author allotted a prearranged amount of time to identify and document threats. Because of this, there is no guarantee that all possible threats and risks are discovered. Furthermore, the analysis applies to a snapshot of the current state of the modeled architecture (based on the architecture information provided by the customer) at the examination time.

Report Distribution

Distribution of this report (in full or in part like diagrams or risk findings) requires that this disclaimer as well as the chapter about the TARALIZER toolkit and method used is kept intact as part of the distributed report or referenced from the distributed parts.